

Белорусский Государственный Университет
Факультет прикладной математики и информатики

Компьютерная графика

Кулинкович Виктория Александровна



Основные алгоритмы вычислительной геометрии. Геометрические задачи визуализации.

- Геометрические преобразования на плоскости
  - способы задания отрезков
  - классификация точки относительно отрезка
  - расстояние от точки до прямой
  - нахождение точки пересечения двух отрезков
  - уравнение нормали
- Отсечение отрезков
- Алгоритм Сазерленда-Коэна
- Алгоритм средней точки
- Алгоритм Лианга-Барски
- Алгоритм Кируса-Бека
- Отсечение многоугольников

Способы задания отрезка

Отрезок (P_1,P_2)

$P_1=(x_1,y_1)$ $P_2=(x_2,y_2)$

1. $Ax+By+C=0$ – уравнение прямой на плоскости

(A,B) – вектор нормали

2. $y=kx+b$ – уравнение прямой на плоскости

$к=(y_2-y_1)/(x_2-x_1)$– угловой коэффициент

3. Параметрическое задание отрезка:

$P=P+t(P_2-P_1),\ 0 \le t \le 1$

$x=x_1+t(x_2-x_1),\ 0 \le t \le 1$

$y=y_1+t(y_2-y_1),\ 0 \le t \le 1$

Скалярное произведение векторов

Скалярным произведением двух векторов называется число, равное произведению модулей этих векторов на косинус угла между ними.

$P_1=(x_1,y_1)$ $P_2=(x_2,y_2)$
$P_3=(x_3,y_3)$ $P_4=(x_4,y_4)$

$$P_1P_2 \cdot P_3P_4 = |P_1P_2||P_3P_4|\cos(\varphi)$$

$$P_1P_2 \cdot P_3P_4 = (x_2 - x_1)(x_4 - x_3) + (y_2 - y_1)(y_4 - y_3)$$

Классификация точки относительно отрезка

Одна из важнейших операций заключается в определении положения точки относительно направленного отрезка прямой линии. Результат выполнения операции показывает, лежит ли точка слева или справа от направленного отрезка прямой линии или, если она совпадает с прямой линией, располагается она после конца направленного отрезка этой прямой линии, до начала, а если ни то, ни другое, то не совпадает ли она с началом или с концом или лежит между ними. Направленный отрезок прямой линии четко разделяет плоскость на семь непересекающихся областей.

Классификация точки относительно отрезка



Нахождение точки пересечения двух отрезков

Пересечение отрезков:

a) Пустое (не пересекаются)
b) Одна точка (пересекаются)
c) Отрезок (накладываются друг на друга)



Нахождение точки пересечения двух отрезков

1. Точка пересечения двух прямых, заданных в общем виде

$$\begin{cases} A_1x+B_1y+C_1=0 \\ A_2x+B_2y+C_2=0 \end{cases}$$

2. Точка пересечения прямой и отрезка, заданного параметрически.

$A_1x+B_1y+C_2=0$ - прямая

$P_1=(x_1,y_1)$ $P_2=(x_2,y_2)$ - отрезок

$x=x_1+t(x_2-x_1),\ 0 \le t \le 1$

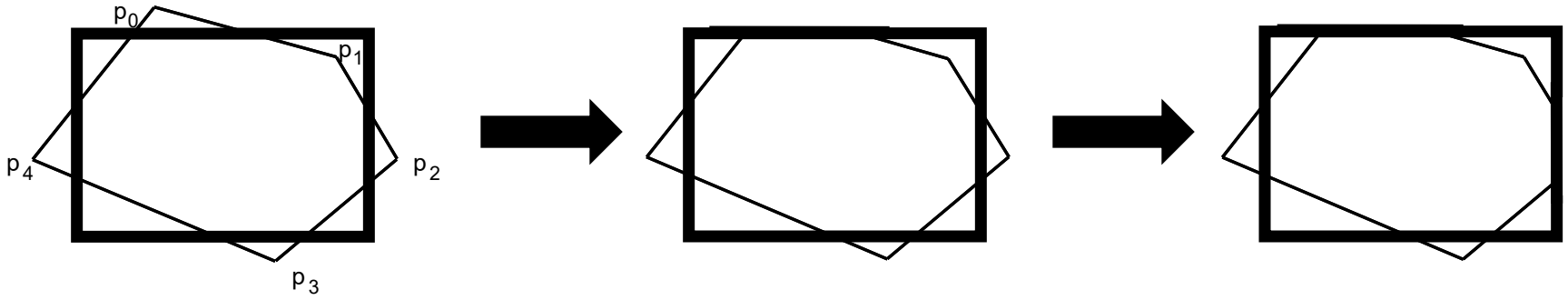
$y=y_1+t(y_2-y_1),\ 0 \le t \le 1$

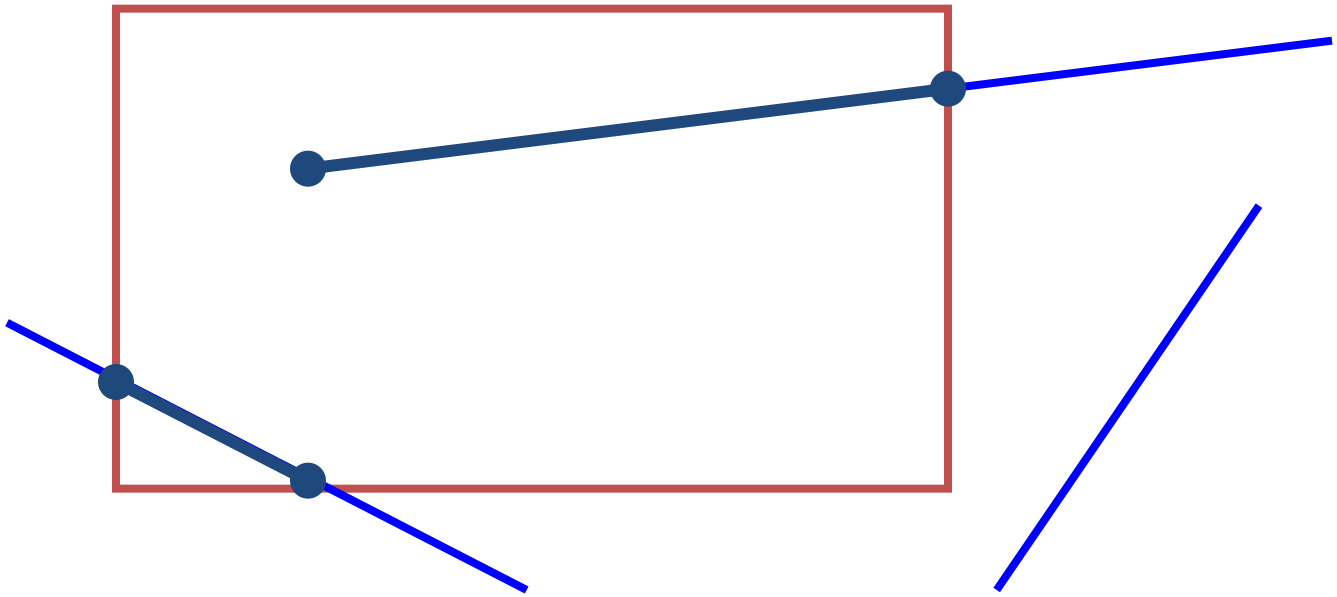
Отсечение

Отсечение (clipping) – это процедура обнаружения и исключения из дальнейшего анализа элементов сцены (линий, поверхностей объектов), которые не попадают в видимую область, отображаемую на экране.

Это операция выполняется одной из первых в процессе визуализации.

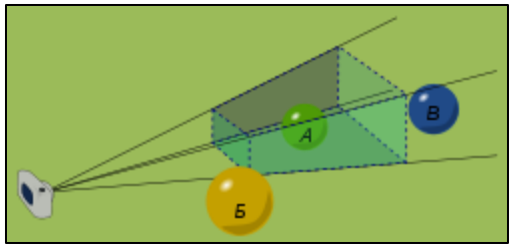
Отсечение

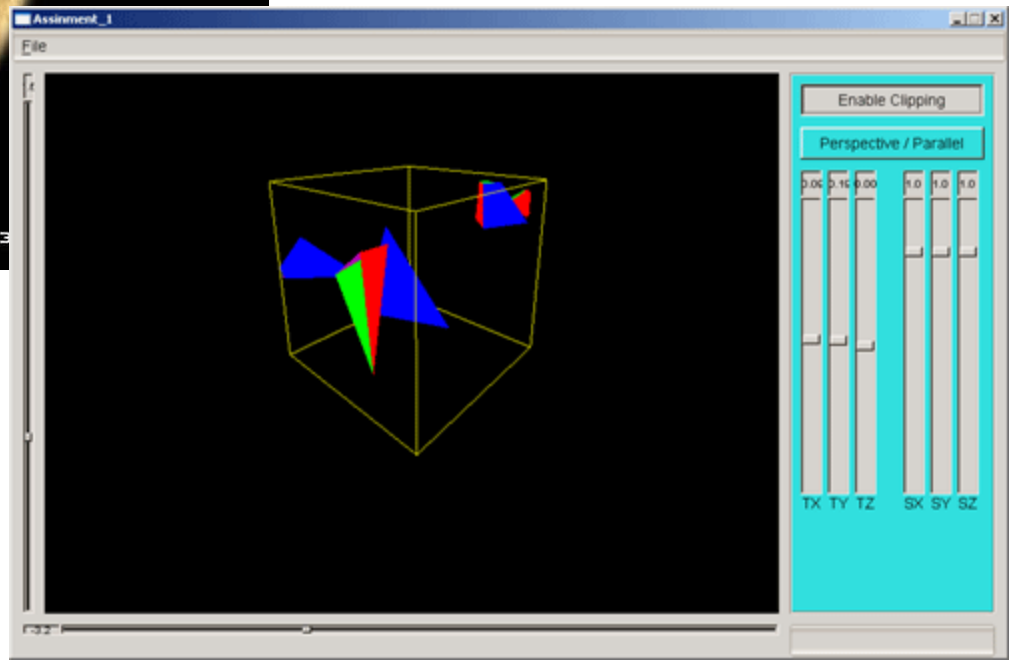




Отсечение


0
100
4:34
$
10


А
Б
В


Assinment_1
File
Enable Clipping
Perspective / Parallel
TX TY TZ
SX SY SZ



Отсечение

Преимущества отсечения непосредственно перед растеризацией:

Можно было бы просто проверять, лежит ли пиксель внутри растра непосредственно перед изменением его цвета. К сожалению, такое решение в большинстве случаев слишком неэффективно. Во-первых, подобная проверка сама по себе чрезвычайно замедлит растеризацию. Во-вторых, в тех случаях, когда значительная часть объекта лежит вне растра, растеризация для этой области будет выполняться вхолостую и отсечение подобных областей опять-таки ускорит вычисления.



Методы отсечения

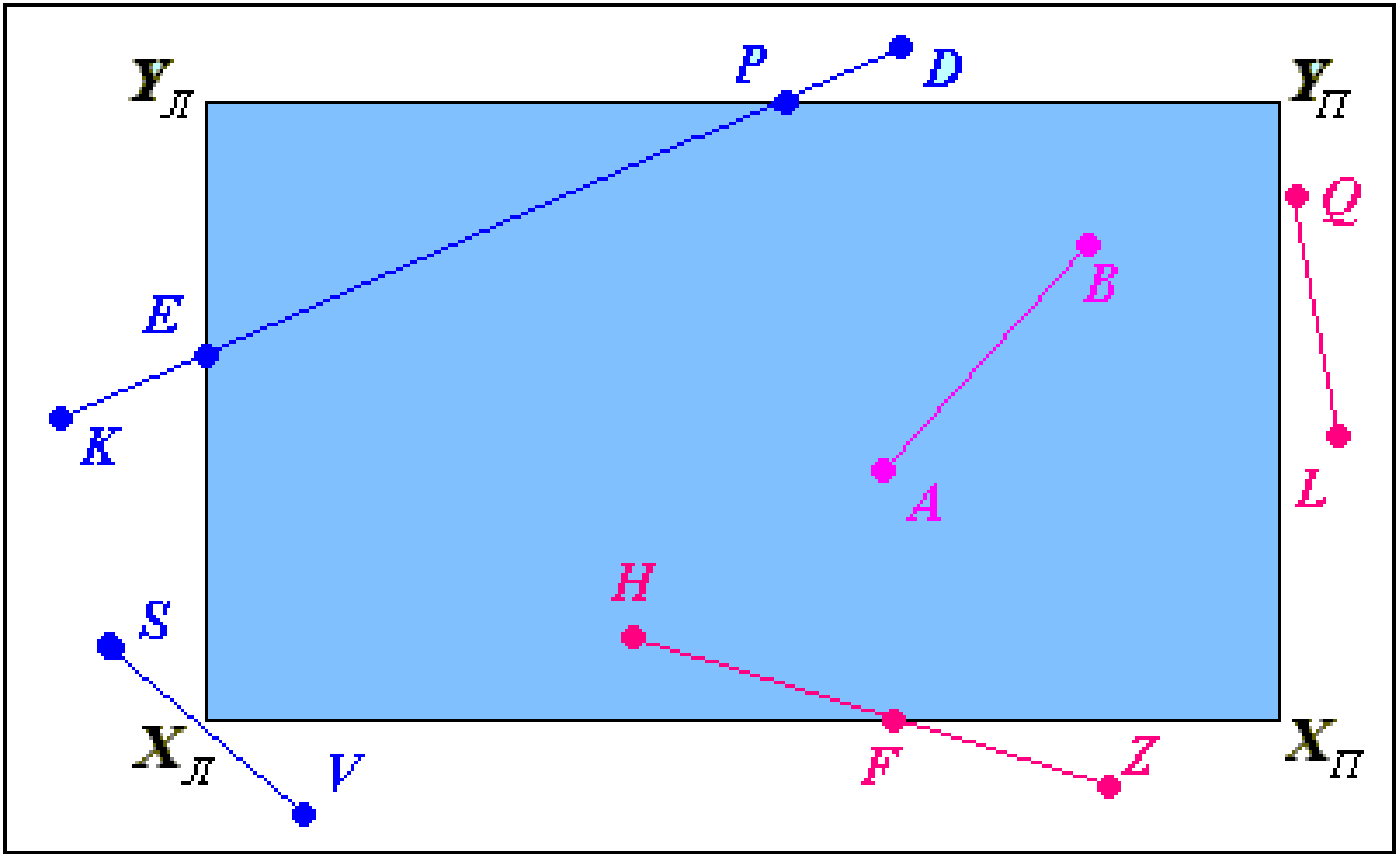
- 2D и 3D.
- Отсечение отрезков, многоугольников, сложных объектов.
- Реализованные аппаратно и программно.

Отсечение отрезков. Алгоритм Сазерленда-Коэна

Дэн Коэн, Айвен Сазерленд, 1966-1968.

Алгоритм Сазерленда-Коэна осуществляет двумерное отсечение отрезка относительно прямоугольника со сторонами, параллельными координатным осям.

Отсечение отрезков. Алгоритм Сазерленда-Коэна



Отсечение отрезков. Алгоритм Сазерленда-Коэна

$(x_{\min}, y_{\min}), \quad (x_{\max}, y_{\max})$ - координаты нижнего левого и верхнего правого углов отсекающего прямоугольника.

Точки, лежащие внутри окна:

$$x_{\min} \le x \le x_{\max}$$

$$y_{\min} \le y \le y_{\max}$$

Если неравенства не выполняются => точка отсекается.

* Отсечение отдельных точек применяется довольно редко.

Отсечение отрезков. Алгоритм Сазерленда-Коэна

Плоскость экрана делится на 9 областей, каждой из которых приписывается четырехбитный код – *характеристический код (outcode)* – $(b_0 b_1 b_2 b_3)$

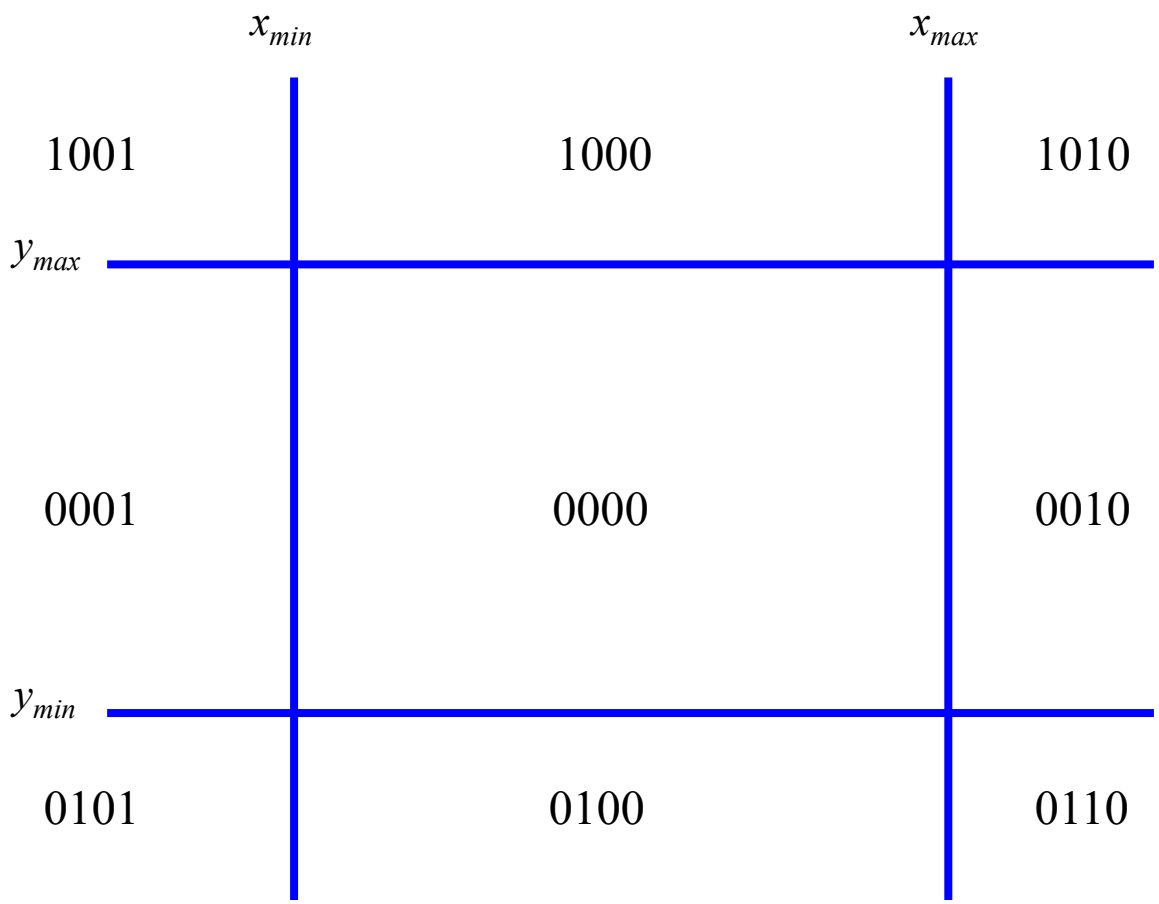
$$b_0 = \begin{cases} 1, & y > y_{\max} \\ 0, & otherwise \end{cases}$$

$$b_1 = \begin{cases} 1, & y < y_{\min} \\ 0, & otherwise \end{cases}$$

$$b_2 = \begin{cases} 1, & x > x_{\max} \\ 0, & otherwise \end{cases}$$

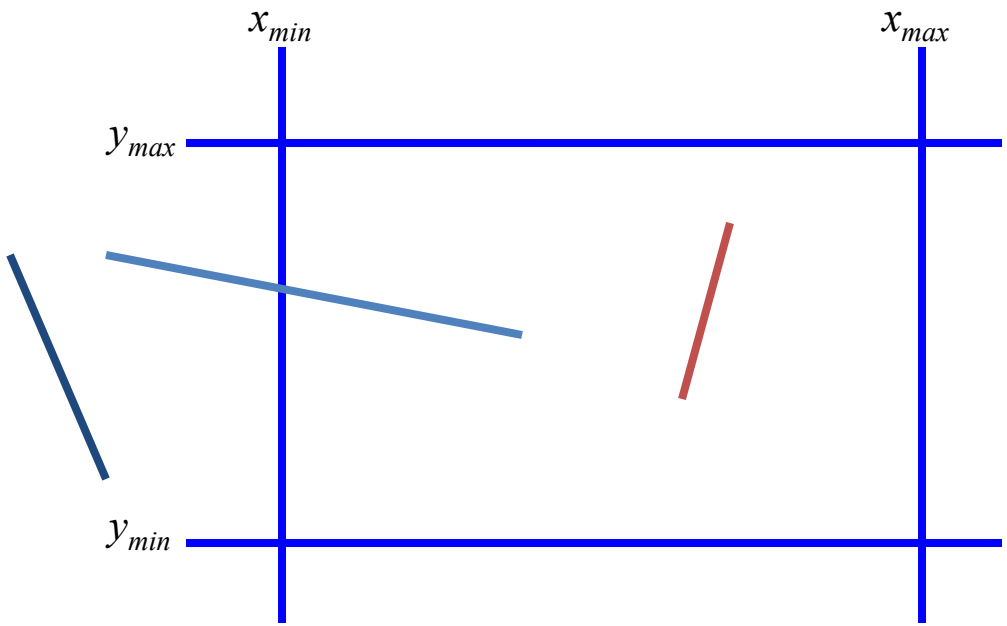
$$b_3 = \begin{cases} 1, & x < x_{\min} \\ 0, & otherwise \end{cases}$$

Отсечение отрезков. Алгоритм Сазерленда-Коэна



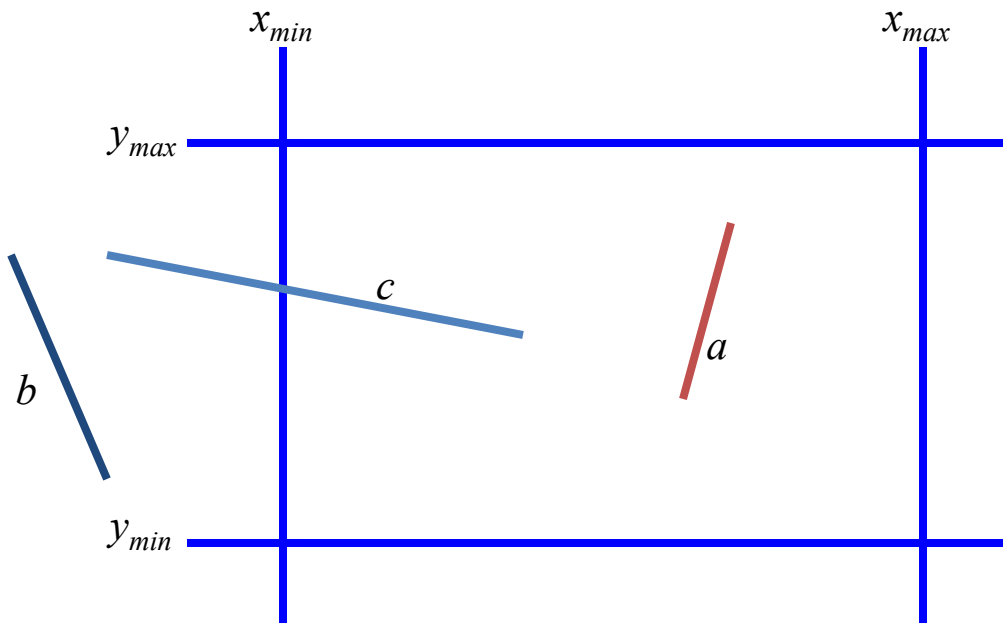
Отсечение отрезков. Алгоритм Сазерленда-Коэна

Пусть O_1 и O_2 – характеристические коды конечных точек рассматриваемого отрезка



Отсечение отрезков. Алгоритм Сазерленда-Коэна

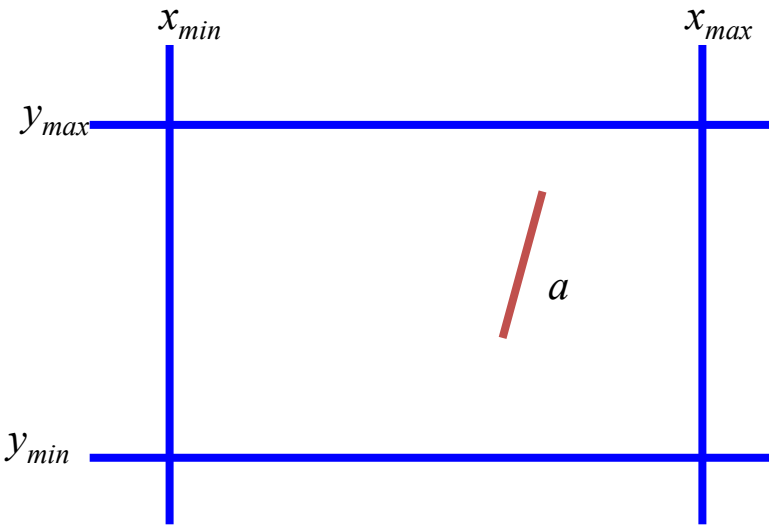
Рассмотрим тривиальные случаи и первым делом отбросим полностью видимые (такие как *a*) и полностью невидимые (такие как *b*) отрезки

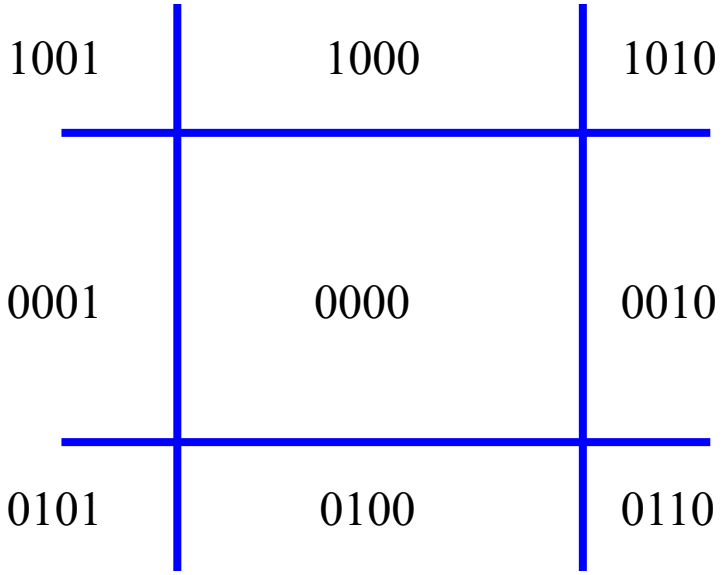


Отсечение отрезков. Алгоритм Сазерленда-Коэна

1. Отрезок лежит целиком внутри отсекающего окна (будет полностью видимым) : $O_1 = O_2 = 0000$.

   Отрезок передается для дальнейшей растеризации.

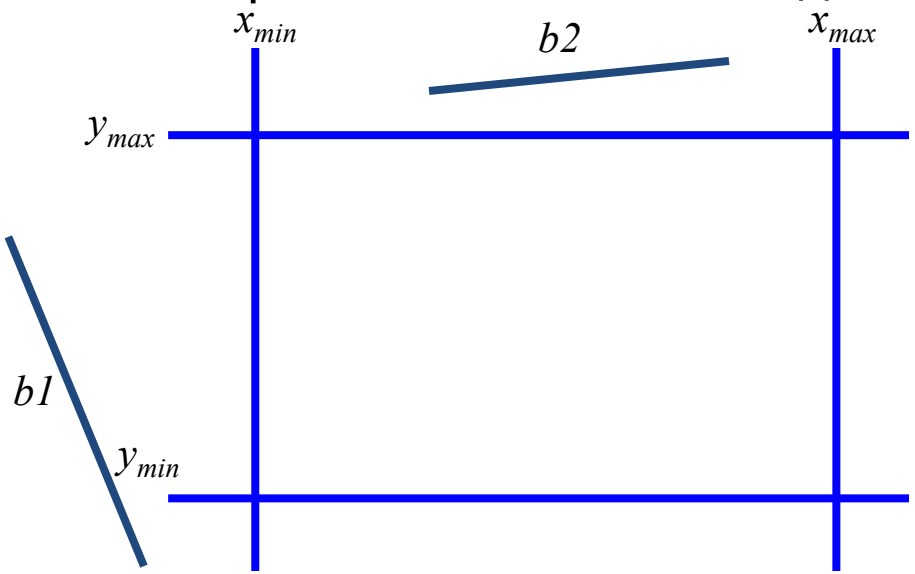


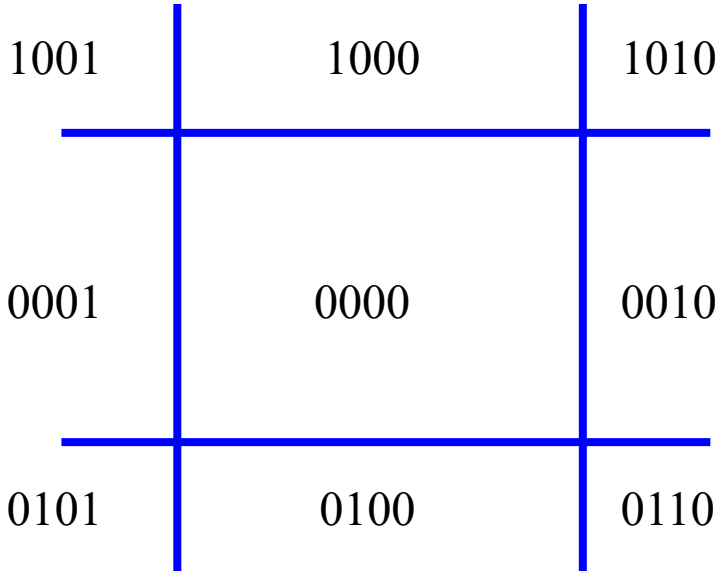


Отсечение отрезков. Алгоритм Сазерленда-Коэна

2. Обе конечные точки лежат по одну сторону от какой-либо границы отсекающего окна (отрезок будет полностью невидимым) => O_1 & O_2 ≠ 0000

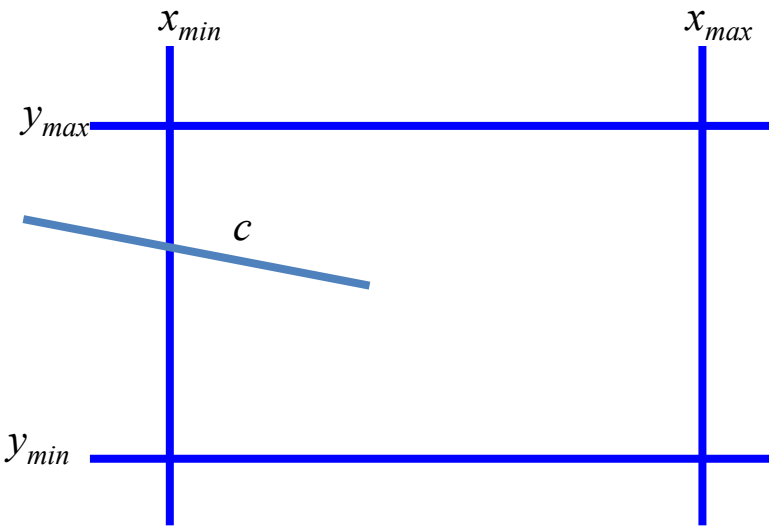
   Отрезок исключается из дальнейшего рассмотрения

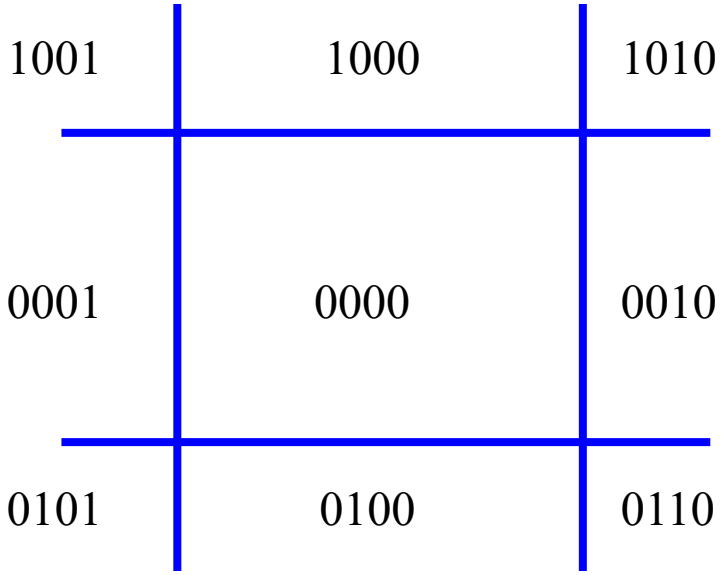




Отсечение отрезков. Алгоритм Сазерленда-Коэна

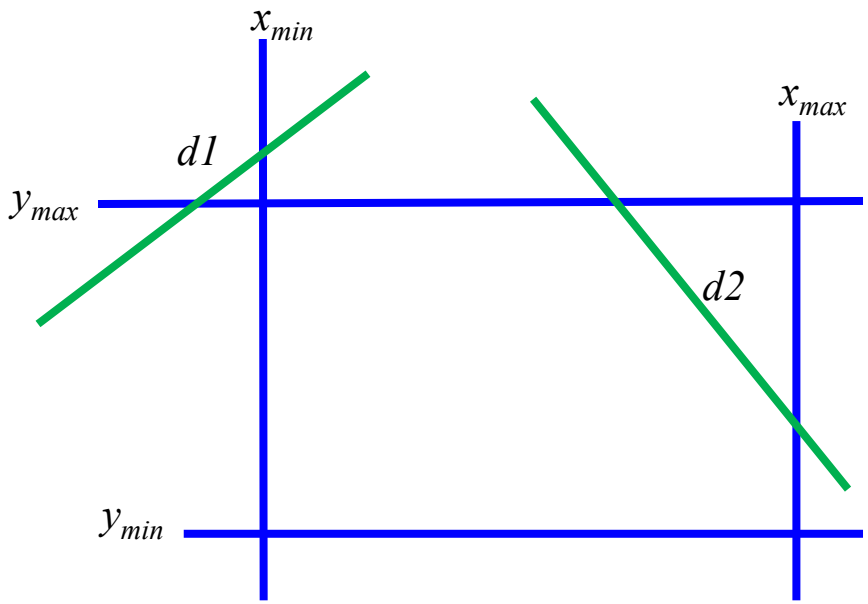
3. Одна из конечных точек лежит внутри отсекающего окна, другая вне его (отрезок будет частично видимым) :
   O_1 =0000 и $O_2 \neq 0000$ либо $O_1 \neq 0000$ и $O_2 = 0000$.
   Отрезок передается для дальнейшей обработки.

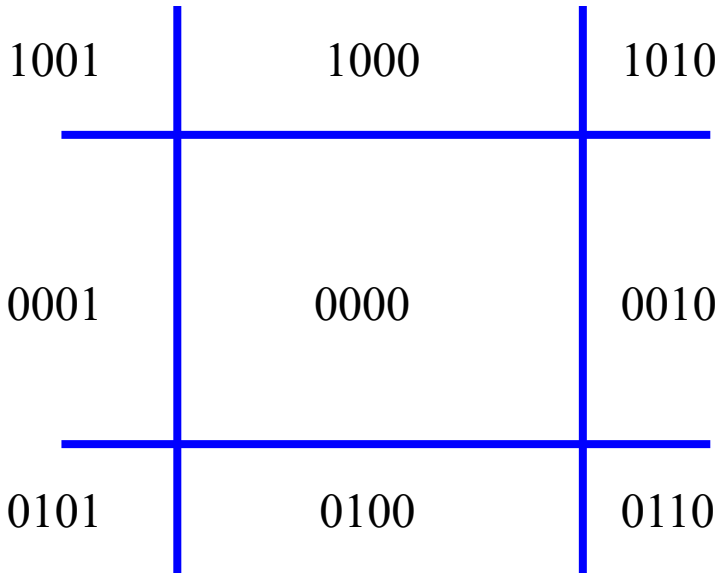




Отсечение отрезков. Алгоритм Сазерленда-Коэна

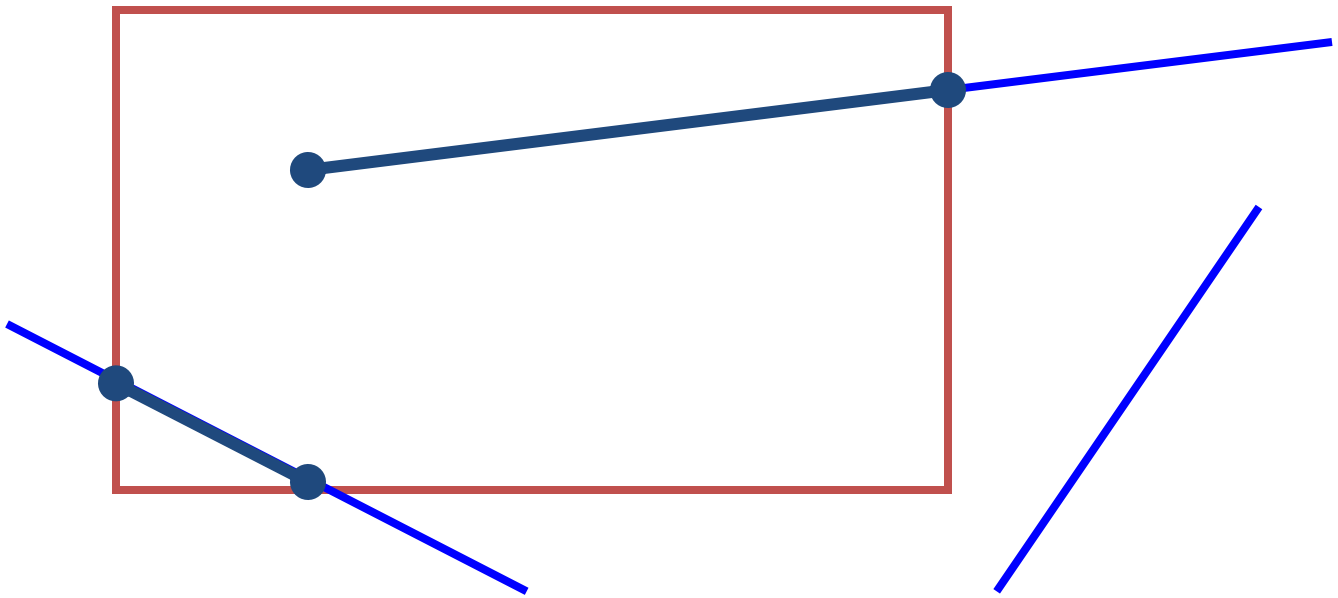
4. Обе конечные точки лежат вне рамки отсечения, но по разные стороны от двух ее границ => O_1 & O_2 = 0000. Отрезок передается для дальнейшей обработки.





Отсечение отрезков. Алгоритм Сазерленда-Коэна

После определения и отбрасывания целиком видимых и целиком невидимых отрезков, решаем задачу вычисления пересечения отрезков со сторонами отсекающего окна.



Отсечение отрезков. Алгоритм Сазерленда-Коэна

Обработка частично видимых отрезков.

1. Если начальная точка P_1 вне окна, то продолжить выполнение, иначе поменять местами P_1 и P_2.
2. Проанализировать код точки P_1 для определения стороны окна, с которой есть пересечение (по положению единичного бита).
3. Выполнить расчет пересечения: P^*_1
4. Заменить точку P_1 на точку пересечения P^*_1

$$P_1P_2 \Rightarrow P_1^*P_2$$

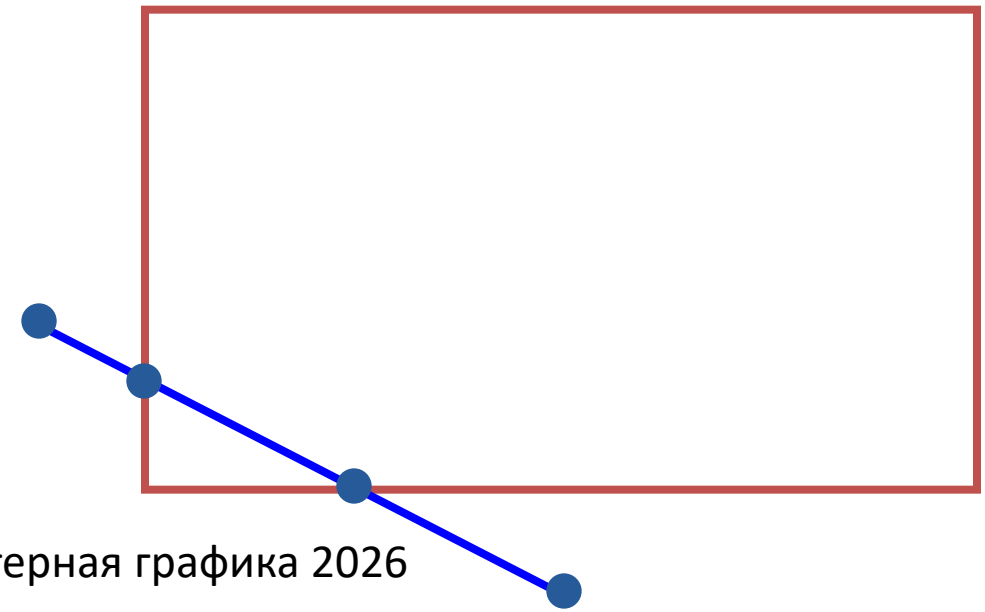
5. Если полученный отрезок полностью видимый – передать его для дальнейшей обработки, иначе перейти к пункту 1.

Отсечение отрезков. Алгоритм Сазерленда-Коэна

$P_1=(x_1,y_1)$ $P_2=(x_2,y_2)$

$y = k(x-x_1)+y_1$ $y = k(x-x_2)+y_2$

$K=(y_2-y_1)/(x_2-x_1)$



Отсечение отрезков. Алгоритм Сазерленда-Коэна

Расчет точки пересечения со сторонами отсекающего окна.

- Пересечение с верхней горизонтальной границей: $y=y_{max}$

$$x = x_1 + (1/k)(y_{max} - y_1) \quad y = y_{max}$$

- Пересечение с нижней горизонтальной границей: $y=y_{min}$

$$x = x_1 + (1/k)(y_{min} - y_1) \quad y = y_{min}$$

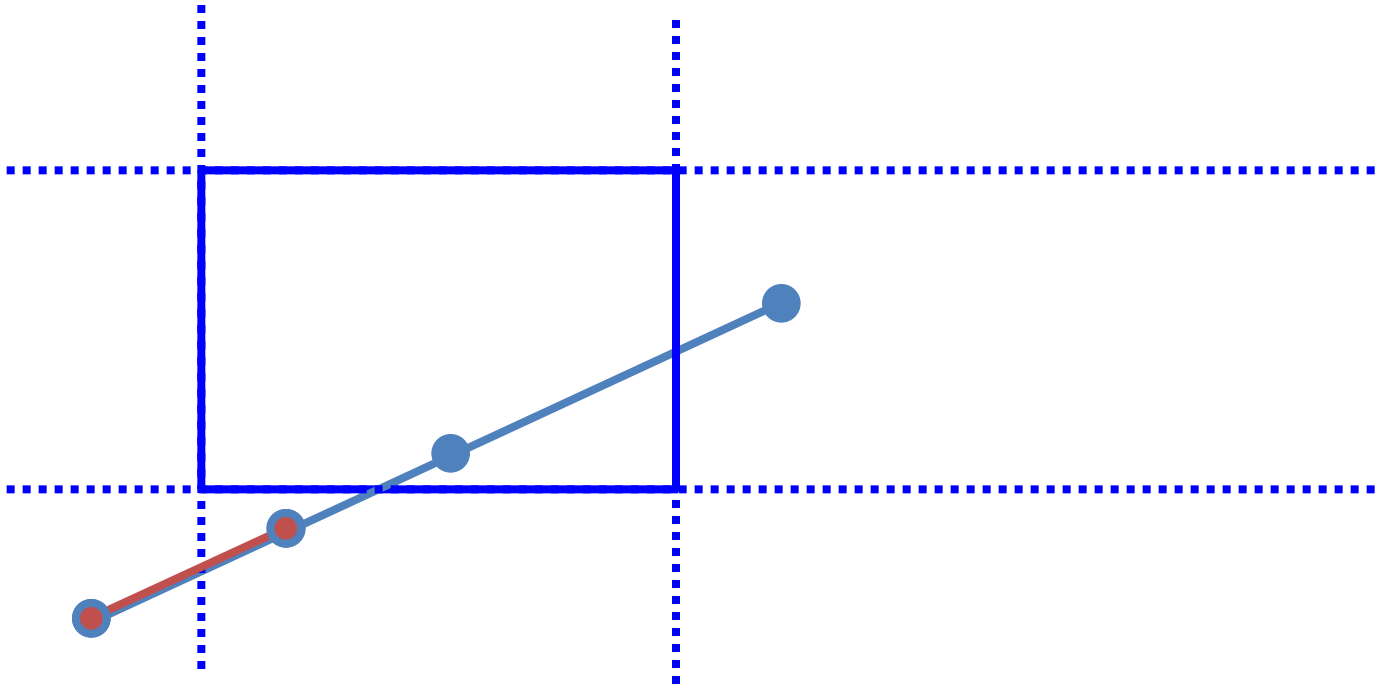
- Пересечение с правой вертикальной границей: $x=x_{max}$

$$y = y_1 + k(x_{max} - x_1) \quad x = x_{max}$$

- Пересечение с левой вертикальной границей: $x=x_{min}$

$$y = y_1 + k(x_{min} - x_1) \quad x = x_{min}$$

Отсечение отрезков. Алгоритм средней точки



Отсечение отрезков. Алгоритм средней точки

```
Отсечь( отрезок AB )
{
    if( длина AB меньше размера пикселя )
                return;
    if( AB лежит вне отсекающего прямоугольника
    )                    return;
    if( AB лежит внутри отсекающего
    прямоугольника )
                { Отобразить AB; return; }
    Отсечь (A, (A+B)/2);
    Отсечь ((A+B)/2),B);
}
```

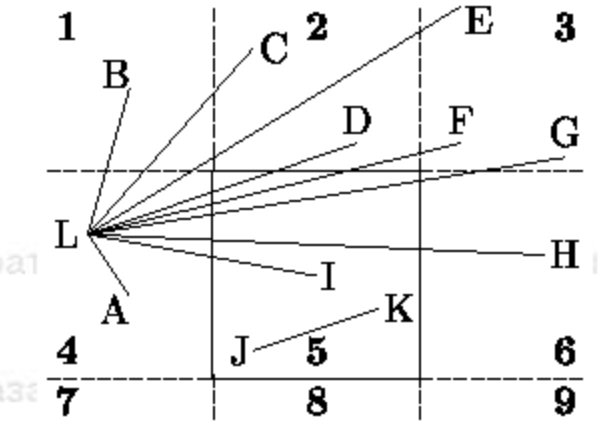
Отсечение отрезков. Алгоритм средней точки

Алгоритм средней точки прост, но не очень эффективен на практике, так как требует большой глубины рекурсии. В большинстве случаев алгоритм Сазерленда-Коэна предпочтительнее. Прием с делением отрезка пополам может оказаться эффективным при отсечении относительно сложной непрямоугольной области, для которой проверить принадлежность точки к окну существенно проще, чем найти пересечение отрезка с границей. Алгоритм средней точки также обобщается на случай трехмерного пространства тривиальным образом.

Отсечение отрезков. Двумерный FC-алгоритм

Fast Clipping алгоритм.

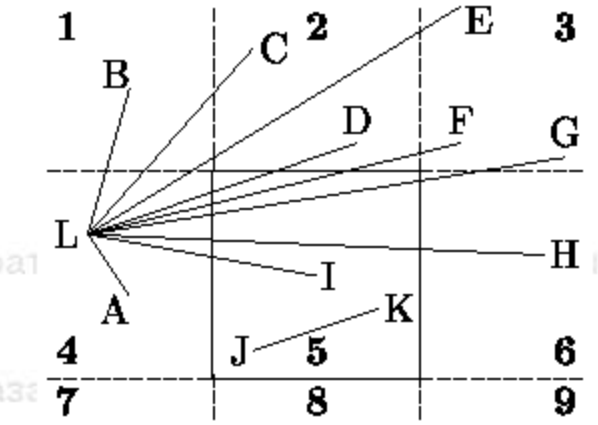
- Использует кодирование, но не конечных точек, а линий целиком.
- Возможен 81 вариант расположения отрезка.
- Реализация громоздкая, но сам алгоритм быстрый.



Отсечение отрезков. Двумерный FC-алгоритм

Fast Clipping алгоритм.

- Использует кодирование, но не конечных точек, а линий целиком.
- Возможен 81 вариант расположения отрезка.
- Реализация громоздкая, но сам алгоритм быстрый.



Отсечение отрезков. Алгоритм Лианга-Барски (Ляна-Барски)

Ю-Донг Лианг и Брайан Барски, 1984.

Использует параметрическое представление отрезка.

Отсекающее окно – прямоугольник со сторонами, параллельными осям координат; (x_{min},y_{min}) – нижний левый угол; (x_{max},y_{max}) – верхний правый угол.

На 25-62% быстрее алгоритма Сазерленда-Коэна.

Отсечение отрезков. Алгоритм Лианга-Барски

- Использует параметрическое представление отрезка.
- Отсекающее окно – прямоугольник со сторонами, параллельными осям координат; (x_{min},y_{min}) – нижний левый угол; (x_{max},y_{max}) – верхний правый угол.
- На 25-62% быстрее алгоритма Сазерленда-Коэна.

$$P_1=(x_1,y_1) \qquad P_2=(x_2,y_2)$$

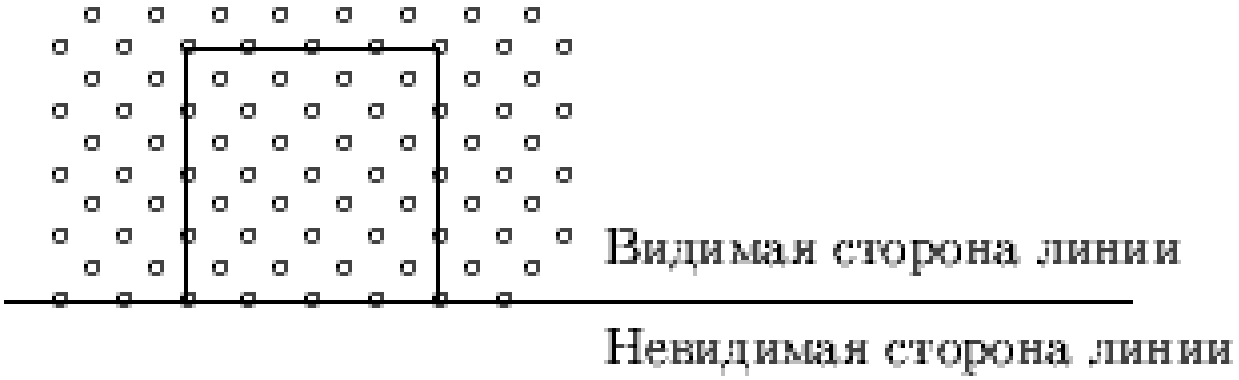
$$P(t)=P_1+t(P_2-P_1),\ 0 \le t \le 1$$

$$x=x_1+t(x_2-x_1),\ 0 \le t \le 1$$

$$y=y_1+t(y_2-y_1),\ 0 \le t \le 1$$

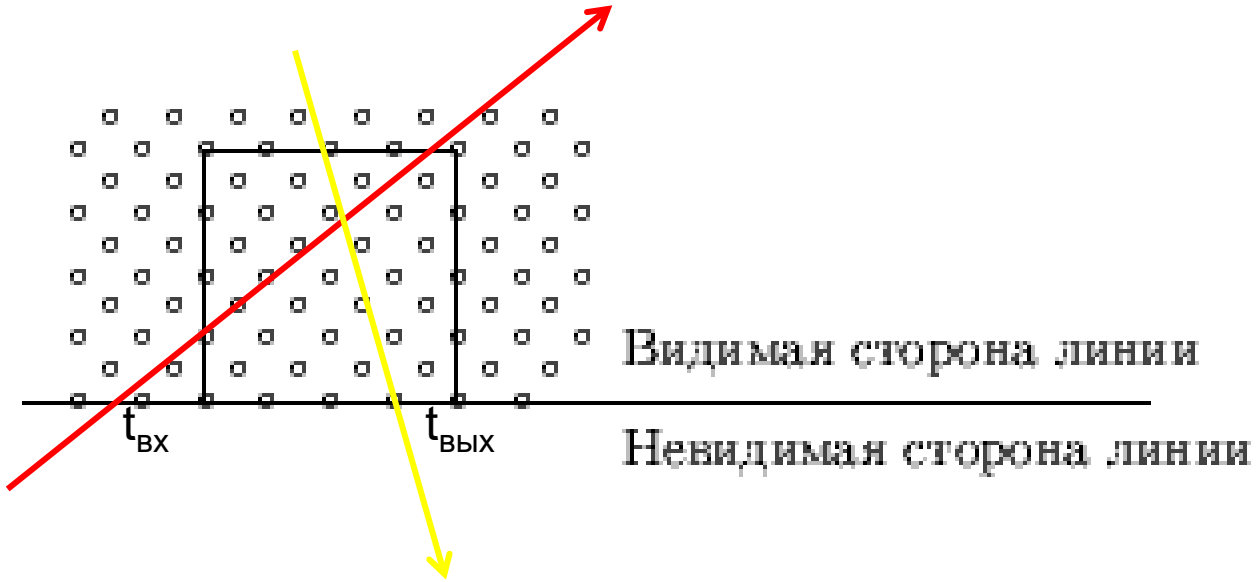
Отсечение отрезков. Алгоритм Лианга-Барски

Каждая из прямых, содержащих стороны отсекающего прямоугольника, делит плоскость на 2 области: видимую и невидимую.



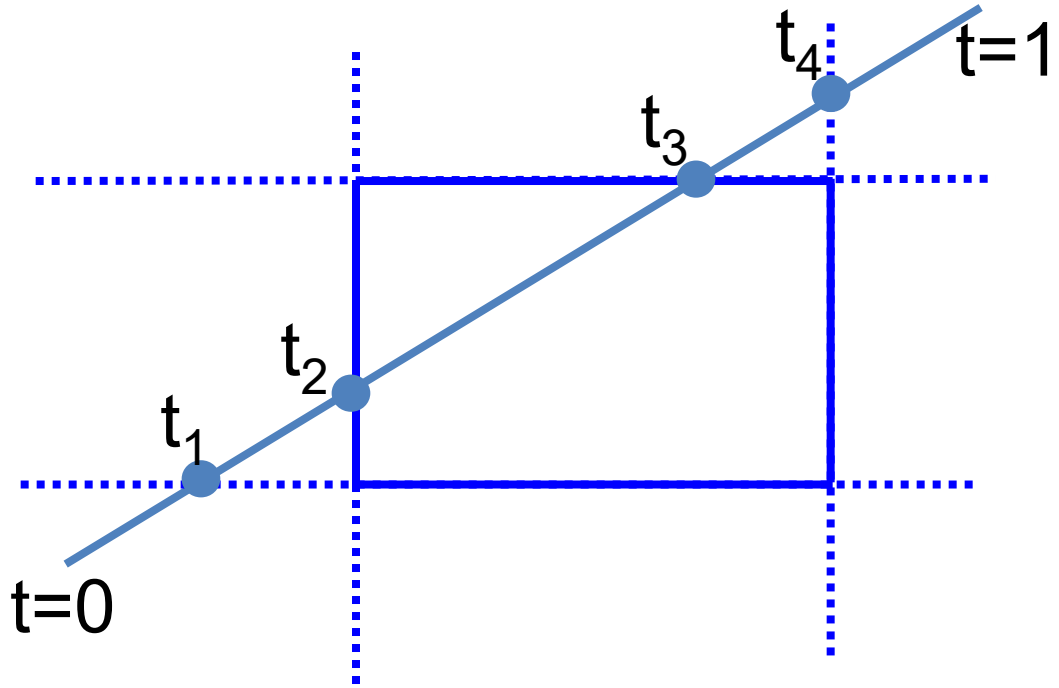
Соответственно, точка пересечения со стороной прямоугольника (или с ее продолжением) классифицируется как «входящая», если начальная точка отрезка расположена в «невидимой» относительно данной стороны области, а конечная – в «видимой», и как «выходящая» в противном случае.

Отсечение отрезков. Алгоритм Лианга-Барски



Для того, чтобы получить истинные точки пересечения отрезка со сторонами окна, необходимо выбрать максимальное значение параметра t из всех «входящих», и минимальное значение параметра t из всех «выходящих».

Отсечение отрезков. Алгоритм Лианга-Барски



Отсечение отрезков. Алгоритм Лианга-Барски

- Пересечение с нижней горизонтальной границей: $y=y_{min}$

$$t_1 = \frac{y_{\min} - y_1}{y_2 - y_1}$$

- Пересечение с левой вертикальной границей: $x=x_{min}$

$$t_2 = \frac{x_{\min} - x_1}{x_2 - x_1}$$

- Пересечение с верхней горизонтальной границей: $y=y_{max}$

$$t_3 = \frac{y_{\max} - y_1}{y_2 - y_1}$$

- Пересечение с правой вертикальной границей: $x=x_{max}$

$$t_4 = \frac{x_{\max} - x_1}{x_2 - x_1}$$

Отсечение отрезков. Алгоритм Лианга-Барски

- Следующим этапом является определение того, какие из четырех найденных параметров соответствуют истинным точкам пересечения.
- Значения t за пределами интервала [0;1] соответствуют точкам, лежащим вне исходного отрезка – они отбрасываются сразу.
- В итоге мы должны получить ровно 2 значения параметра t, соответствующих координатам видимой части отрезка.

Отсечение отрезков. Алгоритм Лианга-Барски

Параметрическое задание отрезка:

$x=x_1+t(x_2-x_1)=x_1+t\cdot dx,\ 0 \le t \le 1$

$y=y_1+t(y_2-y_1)=y_1+t\cdot dy,\ 0 \le t \le 1$ (1)

Неравенства для точек отсекающего окна

$x_{min} \le x \le x_{max}$

$y_{min} \le y \le y_{max}$ (2)

Подставляя (2) в (1), получаем:

$x_{min} \le x_1+t\cdot dx \le x_{max}$

$y_{min} \le y_1+t\cdot dy \le y_{max}$ (3)

Отсечение отрезков. Алгоритм Лианга-Барски

$$x_{min} \le x_1 + t \cdot dx \le x_{max}$$
$$y_{min} \le y_1 + t \cdot dy \le y_{max} \qquad (3)$$

$$-t \cdot dx \le x_1 - x_{min}$$
$$t \cdot dx \le x_{max} - x_1$$
$$-t \cdot dy \le y_1 - y_{min} \qquad (4)$$
$$t \cdot dy \le y_{max} - y_1$$

$$t \cdot S_2 \le Q_2$$
$$t \cdot S_4 \le Q_4$$
$$t \cdot S_1 \le Q_1 \qquad (5)$$
$$t \cdot S_3 \le Q_3$$

$$t \cdot S_i \le Q_i,\ i = 1,2,3,4$$

Вводятся следующие обозначения:

$S_1 = -dy$ $\quad S_2 = -dx$
$S_3 = dy$ $\quad S_4 = dx$

$Q_1 = y_1 - y_{min}$
$Q_2 = x_1 - x_{min}$
$Q_3 = y_{max} - y_1$
$Q_4 = x_{max} - x_1$

Отсечение отрезков. Алгоритм Лианга-Барски

$$t \cdot S_i \le Q_i,\ i = 1,2,3,4$$

$Q_1 = y_1 - y_{min}$ $Q_2 = x_1 - x_{min}$ $Q_3 = y_{max} - y_1$ $Q_4 = x_{max} - x_1$

$S_1 = -dy$ $S_2 = -dx$ $S_3 = dy$ $S_4 = dx$

Знак S_i будет показывать ориентацию отрезка относительно соответствующей границы: если $S_i > 0$, то отрезок «выходит» из видимой области через границу ***i***; если $S_i < 0$, то отрезок «входит» в видимую область через границу ***i***; если $S_i = 0$, то отрезок параллелен данной границе.

Знак Q_i будет показывать, по какую сторону от соответствующей границы находится начальная точка P_1.

$Q_i \ge 0 \rightarrow$ начальная точка находится на видимой стороне

$Q_i < 0 \rightarrow$ начальная точка находится на невидимой стороне

Отсечение отрезков. Алгоритм Лианга-Барски

$t_{вх}=0$; $t_{вых}=1$;
Для каждого i=1,2,3,4:
{Если $S_i>0$ то
$t_{вых}=\min\{Q_i/S_i , t_{вых}\}$
Если $S_i<0$ то =>
$t_{вх}=\max\{Q_i/S_i , t_{вх}\}$
Если $S_i=0$ то :
если $Q_i<0$ то отрезок невидим отн-но границы i, отбрасываем его
если $Q_i\geq 0$ то отрезок полностью видимый отн-но границы i,
переходим к следующему i }

$P_{вх}=P_1+t_{вх}\cdot(P_2-P_1)$ и $P_{вых}=P_1+t_{вых}\cdot(P_2-P_1)$ – результирующие координаты видимой части отрезка, передаваемые для дальнейшей растеризации

Отсечение отрезков. Алгоритм Лианга-Барски

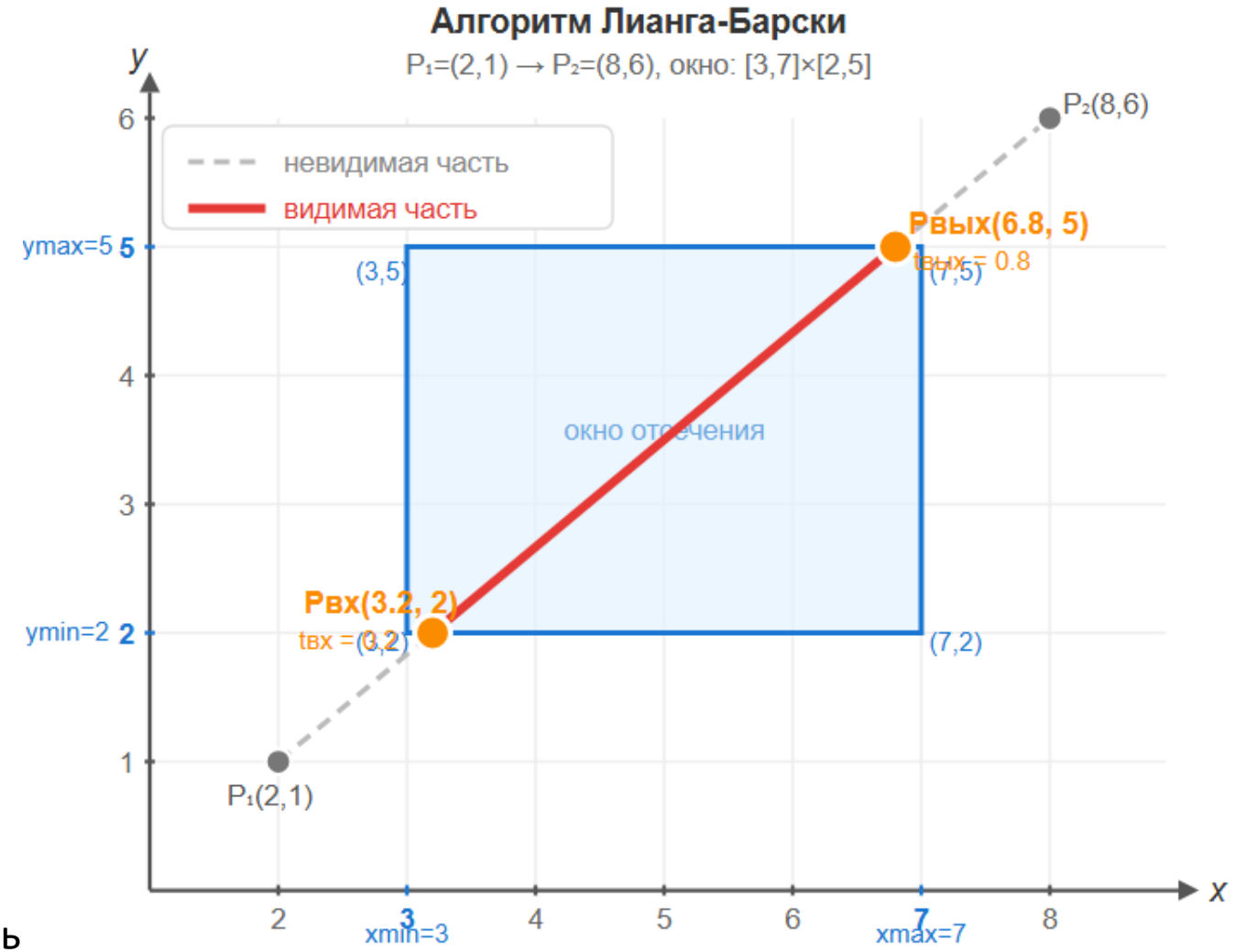
P1=(2,1) P2=(8,6) dx=6 dy=5
Окно: xmin=3, xmax=7, ymin=2, ymax=5

S1=-dy=-5 Q1=y1-ymin= -1 t1=Q1/S1= 0.2 (вход)
S2=-dx=-6 Q2=x1-xmin= -1 t2=Q2/S2= 0.17 (вход)
S3= dy= 5 Q3=ymax-y1= 4 t3=Q3/S3= 0.8 (выход)
S4= dx= 6 Q4=xmax-x1= 5 t4=Q4/S4= 0.83 (выход)

tвх = max(0, 0.2, 0.17) = 0.2
tвых = min(1, 0.8, 0.83) = 0.8
tвх < tвых → отрезок частично видим

Pвх = (2 + 0.2·6, 1 + 0.2·5) = (3.2, 2.0)
Pвых = (2 + 0.8·6, 1 + 0.8·5) = (6.8, 5.0)

Отсечение отрезков. Алгоритм Лианга-Барски



Компь

Отсечение отрезков. Алгоритм Кируса-Бека.

Майкл Кирус и Джей Бек, 1978.

- Обобщение алгоритма Лианга-Барски.
- Использует параметрическое представление отрезка.
- Отсекающее окно – произвольное, выпуклой формы.

$$P(t)=P_1+t(P_2-P_1),\ 0 \le t \le 1$$
$$x=x_1+t(x_2-x_1),\ 0 \le t \le 1$$
$$y=y_1+t(y_2-y_1),\ 0 \le t \le 1$$

Отсечение отрезков. Алгоритм Кируса-Бека.

Общий принцип работы алгоритма:

1. Рёбра отсекающего многоугольника обходятся против часовой стрелки. Пусть текущее ребро – *Ei*, *Ni* – внутренняя нормаль.
2. Для ребра *Ei* вычисляем параметр t_{Ei}, задающий точку пересечения прямой, содержащей ребро *Ei,* с прямой, на которой лежит отрезок *P1P2*.

Отсечение отрезков. Алгоритм Кируса-Бека.

Общий принцип работы алгоритма:

3. Вычисляем скалярное произведение S вектора *P1P2* и внутренней нормали *N* к прямой, содержащей ребро *Ei.*

В зависимости от знака произведения возникает одна из следующих ситуаций:

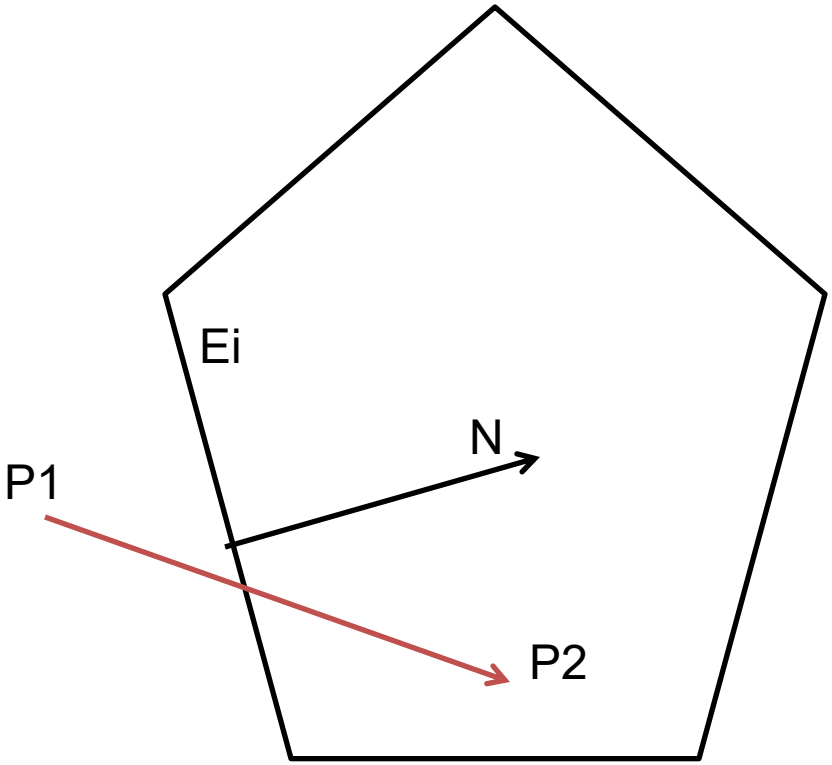
S>0: отрезок входит в окно через ребро Ei;

S<0: отрезок выходит из окна через ребро Ei;

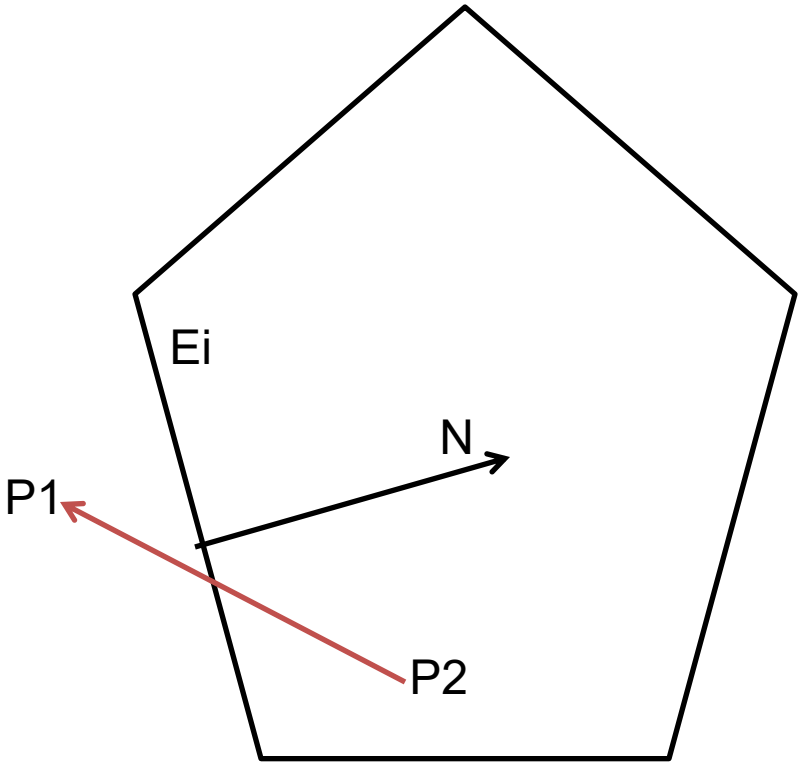
S=0: отрезок параллелен прямой, содержащей ребро Ei;

Отсечение отрезков. Алгоритм Кируса-Бека.

$$\overline{P_1P_2} \cdot \overline{N} > 0$$



$$\overline{P_1P_2} \cdot \overline{N} < 0$$



Отсечение отрезков. Алгоритм Кируса-Бека.

4. Если вычисленный параметр t_{Ei} лежит вне интервала [0;1], то отбрасываем его.

Если вычисленный параметр t_{Ei} лежит в интервале [0;1], а вычисленное скалярное произведение S>0, то записываем t_{Ei} в список *Твх* потенциальных точек входа в отсекающее окно.

Если вычисленный параметр t_{Ei} лежит в интервале [0;1], а вычисленное скалярное произведение S<0, то записываем t_{Ei} в список *Твых* потенциальных точек выхода из отсекающего окна.

Если вычисленное скалярное произведение S=0, то отрезок либо полностью видимый относительно данного ребра, либо полностью невидимый.

Отсечение отрезков. Алгоритм Кируса-Бека.

5. Ищем максимальное значение параметра в списке *Твх* и минимальное значение параметра в списке *Твых.*

Найденные значения и будут искомыми точками пересечения.

В случае, если один из списков окажется пустым, то в результате получим одну точку отсечения.

Если пустым оказался список *Твх,* то внутри отсекающего окна лежит начальная точка отрезка, если же пуст список *Твых*, то внутри отсекающего окна лежит конечная точка.

Если оба списка пустые, то отрезок отбрасывается, как полностью невидимый.

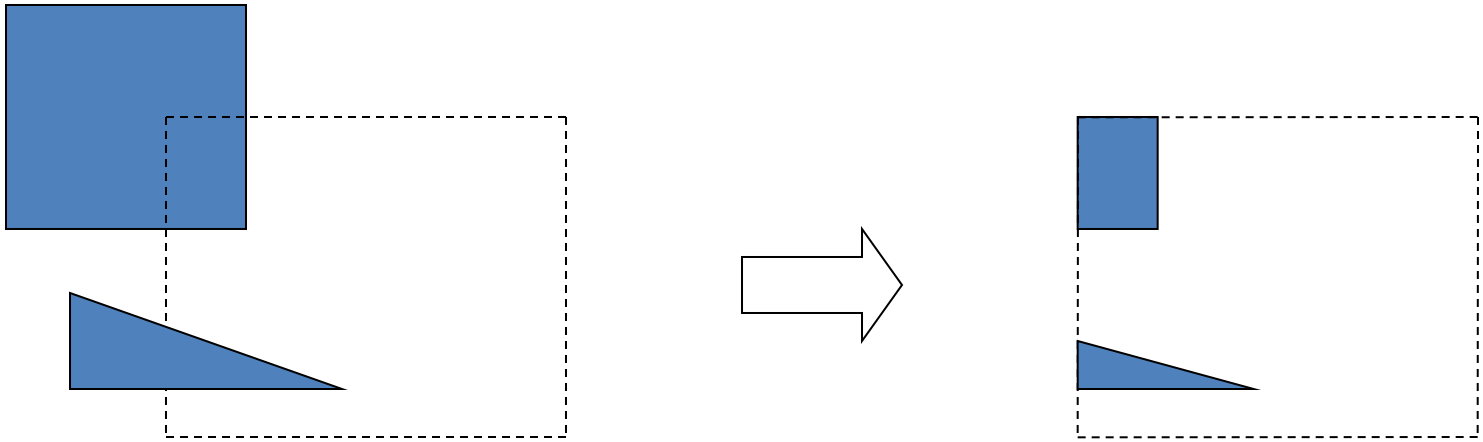
Отсечение отрезков. Сравнение алгоритмов

Алгоритм	Форма окна	Скорость	Когда использовать
Сазерленд-Коэн	Прямоугольник	Базовая	Классика, просто реализовать
FC	Прямоугольник	Максимальная	Много отрезков, нужна скорость
Средней точки	Любое	Низкая	Непрямоугольные окна
Лианг-Барски	Прямоугольник	Высокая	Практический выбор для 2D
Кирус-Бек	Выпуклый многоугольник	Средняя	Произвольное окно

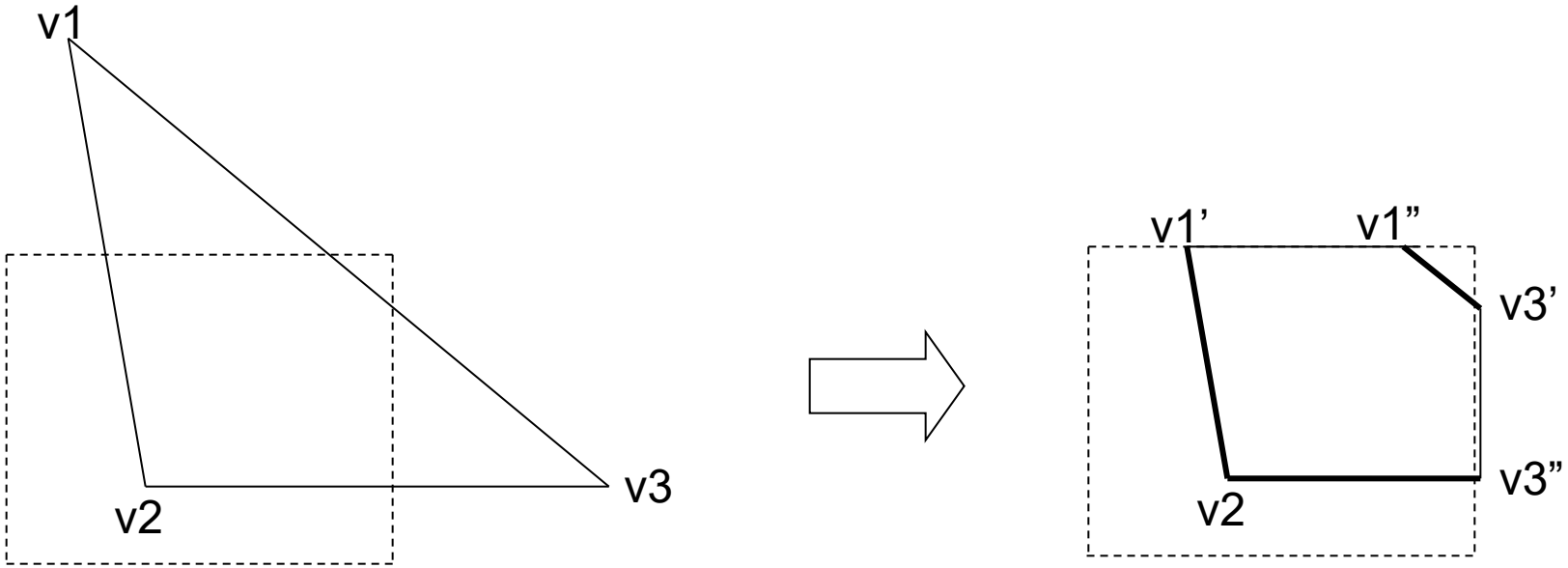
Отсечение многоугольников

Замкнутый многоугольник можно рассматривать как набор отрезков. Тогда к нему применяются стандартные методы отсечения отрезков. В результате получаем снова набор разрозненных отрезков.

Однако чаще многоугольник рассматривается как сплошная область, поэтому необходимо, чтобы замкнутость сохранилась и у результата.



Отсечение многоугольников

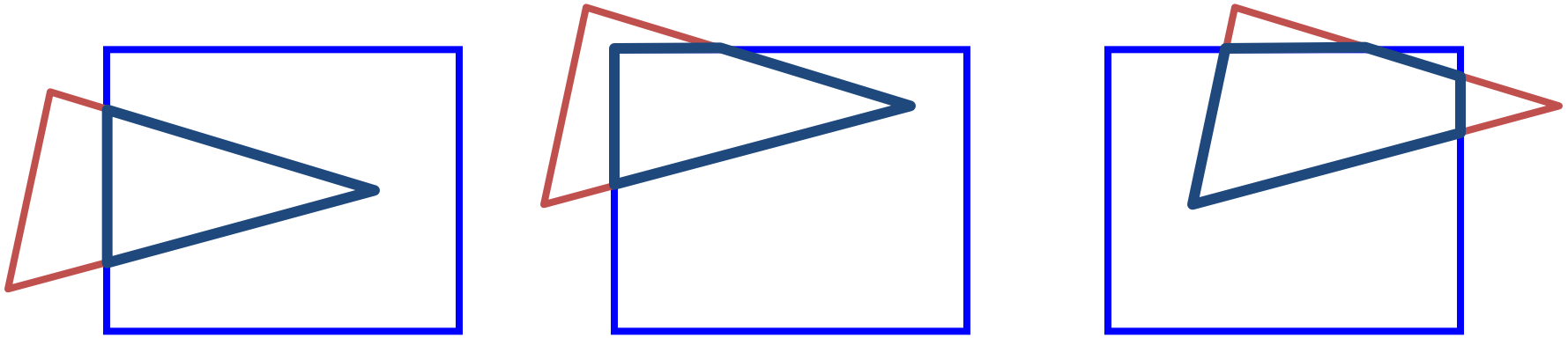


Результат:

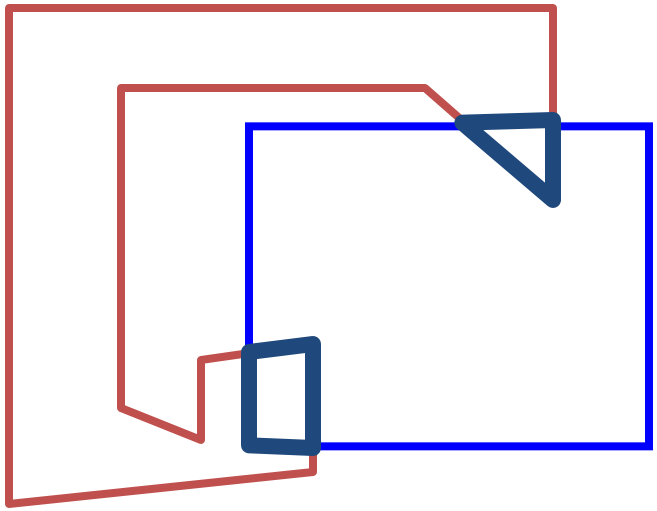
либо набор отрезков v1’v2, v2v3’’,v1’’v3’

либо многоугольник v1’v1’’v3’v3’’v2

Отсечение многоугольников



Отсечение многоугольников





Отсечение многоугольников. Алгоритм Сазерленда-Ходжмана

Sutherland-Hodgman algorithm, 1974
Айвен Сазерленд
Гарри Ходжман

Алгоритм отсечения **выпуклого многоугольника** относительно **выпуклого окна**.

Основная идея алгоритма состоит в том, что отсечь многоугольник относительно одной прямой достаточно просто. Поэтому алгоритм выполняет последовательное отсечение многоугольника каждой из сторон отсекающего окна, на каждом шаге формируя новый список ребер.



Отсечение многоугольников. Алгоритм Сазерленда-Ходжмана

Пример работы алгоритма для отсечения произвольного выпуклого многоугольника прямоугольным отсекающим окном.

- Исходный многоугольник задается списком вершин, который порождает список ребер.

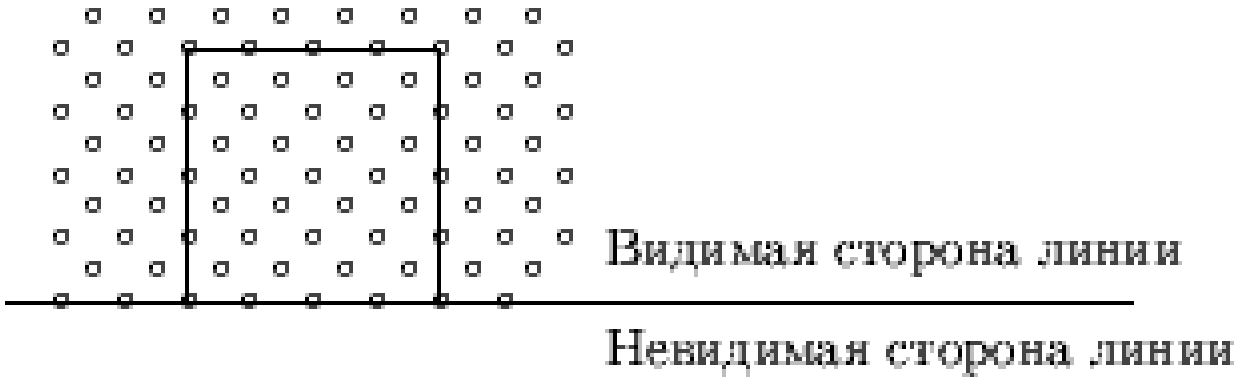
- Отсекаем исходный многоугольник поочередно каждой из сторон окна по следующим правилам, на каждом этапе формируя новый список вершин и соответствующий ему промежуточный многоугольник

Отсечение многоугольников. Алгоритм Сазерленда-Ходжмана

Обрабатываем пересечение каждого ребра исходного многоугольника с текущей стороной отсекающего окна.

В процессе этой обработки в новый список сохраняются некоторые вершины, находящиеся «внутри» относительно этой стороны и некоторые точки пересечения.

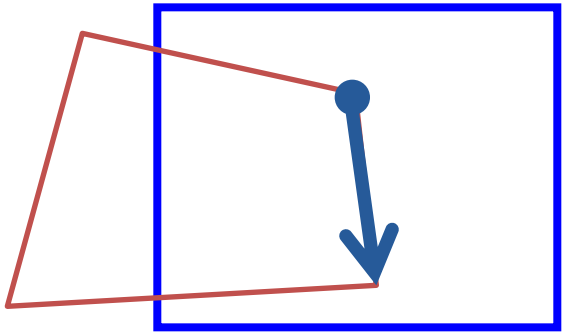
«Внутри» стороны – внутри полуплоскости, отсекаемой этой стороной, то есть на «видимой» области.



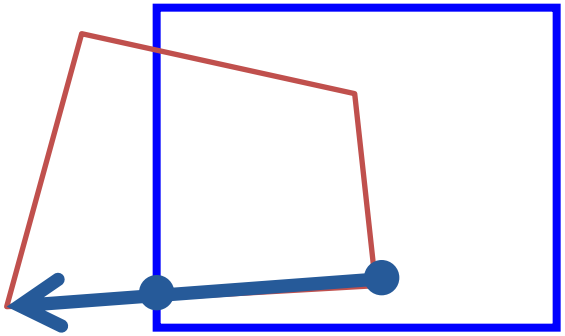
Отсечение многоугольников. Алгоритм Сазерленда-Ходжмана

При такой обработке возможны следующие варианты (пример для пересечения с левой вертикальной границей):

1. Начало и конец ребра многоугольника лежат внутри стороны отсекающего окна; тогда добавляем точку начала к вершинам полигона-результата.

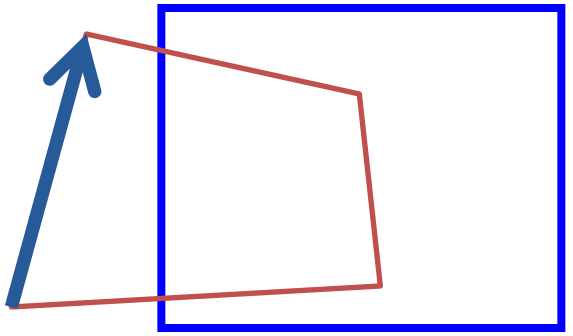


2. Начало лежит внутри стороны отсекающего окна, конец лежит вне; в этом случае вычисляем точку пересечения ребра многоугольника и стороны отсекающего окна; добавляем в список вершин результата начало ребра и вслед за ним точку пересечения.

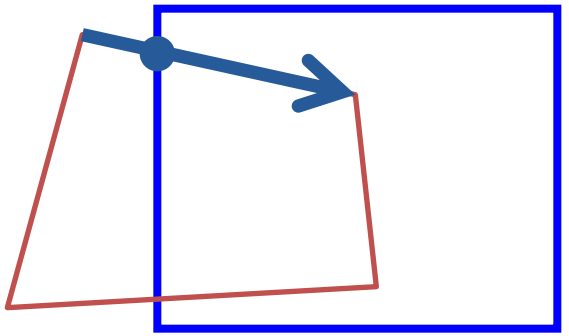


Отсечение многоугольников. Алгоритм Сазерленда-Ходжмана

3. Начало не лежит внутри стороны отсекающего окна, конец тоже; переходим к следующему ребру, никак не изменяя результат.

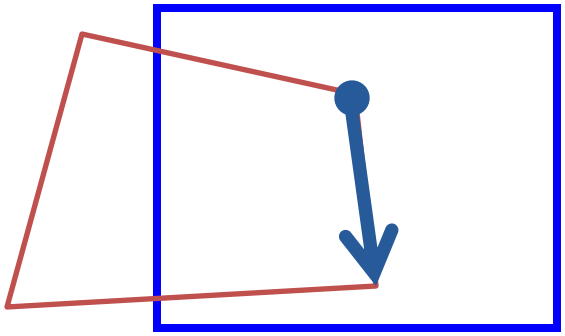


4. Начало не лежит внутри стороны отсекающего окна, конец лежит; считаем точку пересечения и добавляем в новый список только ее.

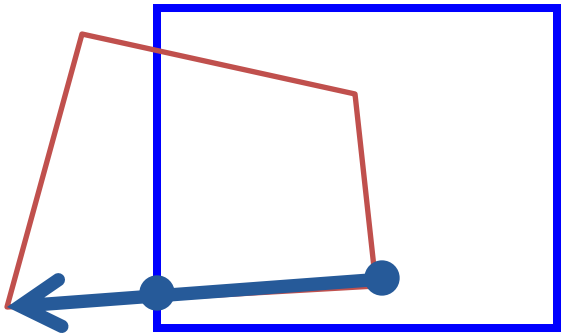


Отсечение многоугольников. Алгоритм Сазерленда-Ходжмана

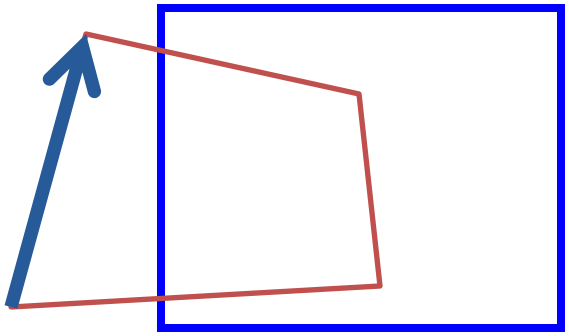
1)



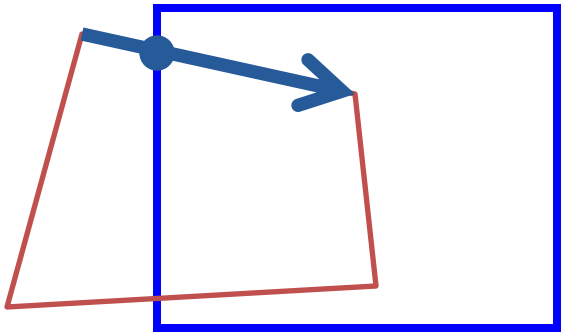
2)



3)



4)





Отсечение многоугольников. Алгоритм Сазерленда-Ходжмана

Частные случаи.

• Точка, лежащая на границе отсекающего окна, считается внутренней точкой (лежащей внутри окна).

• Отрезок, лежащий на одной из границ окна, считается внутренним (лежащим внутри окна).

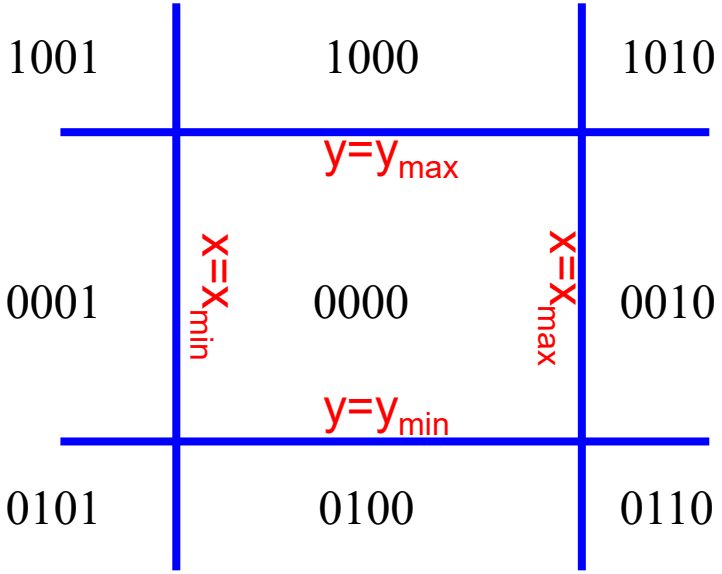
Отсечение многоугольников. Алгоритм Сазерленда-Ходжмана

Особенности реализации.

- Многоугольник задается последовательностью вершин $\{P_1,\ldots,P_n\}$
- Соответствующая последовательность ребер - $\{P_1P_2, P_2P_3,\ldots, P_{n-1}P_n, P_nP_1\}$
- Отсекающее окно задается координатами (x_{min},y_{min}) (x_{max},y_{max})

- Поиск точки пересечения ребра и стороны – просто (рассмотреть все случаи!).
- Определение «видимости» или «невидимости» начальной и конечной точек очередного ребра – по анализу характеристического кода (из алгоритма Сазерленда-Коэна).

Пересечение со стороной $x=x_{min}$:
$b_3 = 1$ => видимая точка, иначе невидимая
Пересечение со стороной $x=x_{max}$:
$b_2 = 1$ => видимая точка, иначе невидимая
Пересечение со стороной $y=y_{min}$:
$b_1 = 1$ => видимая точка, иначе невидимая
Пересечение со стороной $y=y_{max}$:
$b_0 = 1$ => видимая точка, иначе невидимая



Отсечение многоугольников. Алгоритм Сазерленда-Ходжмана

Определение видимости точек.

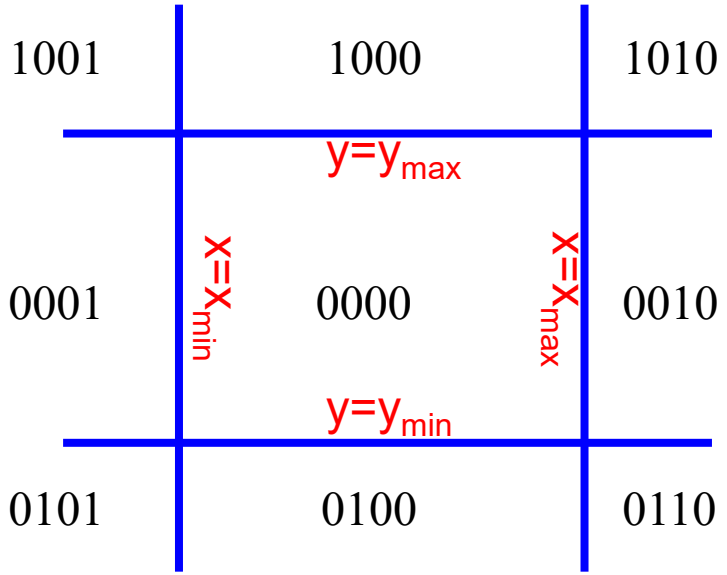
Случай прямоугольного отсекающего окна со сторонами, параллельными координатным осям

Пересечение со стороной $x=x_{min}$:
$b_3 = 1$ => видимая точка, иначе невидимая

Пересечение со стороной $x=x_{max}$:
$b_2 = 1$ => видимая точка, иначе невидимая

Пересечение со стороной $y=y_{min}$:
$b_1 = 1$ => видимая точка, иначе невидимая

Пересечение со стороной $y=y_{max}$:
$b_0 = 1$ => видимая точка, иначе невидимая





Отсечение многоугольников. Алгоритм Сазерленда-Ходжмана

Определение видимости точек.
Случай произвольного выпуклого отсекающего окна: используем нормаль

(Определение видимости точки относительно произвольного ребра — см. алгоритм Кируса-Бека)

От 2D к 3D-отсечению

2D (рассмотрено)	3D (расширение)
Прямоугольное окно	Параллелепипед (view volume)
Сазерленд-Коэн	Cohen–Sutherland 3D (6-битный код)
Лианг-Барски	Liang–Barsky 3D (6 граней)
Сазерленд-Ходжмен	Сазерленд-Ходжмен 3D (отсечение по frustum)